

Rapport de Projet de Fin de Module POO-Java :

Développement d'un Jeu de Dames avec Interface Graphique



Réalisée par :

El Qasemy Niamat
Makhloufi Ouïam
Khaddor Douae



Encadrée par :

Mme. Ibtihaïe Mayouche

Année universitaire : 2024-2025

Table des Matière

TABLE DES MATIERE.....	2
I. INTRODUCTION.....	3
II. PRINCIPE DE JEU	3
III. METHODOLOGIE.....	3
IV. DESCRIPTION SYNTHETIQUE DES CLASSES.....	4
V. LE DIAGRAMME DE CLASSES UML DE L'APPLICATION.....	6
VI. CAPTURES D'ECRAN DE L'EXECUTION DU JEU	7
VII. REPARTITION DES TACHES ENTRE LES MEMBRES.....	9
VIII. CONCLUSION	10

I. Introduction

Dans le cadre du module de Modélisation et Programmation Orientée Objet, nous avons réalisé un projet visant à consolider nos acquis à travers la conception d'une simulation du jeu de dames en Java. Ce projet a pour objectif de renforcer notre maîtrise des principes de la programmation orientée objet, en traduisant la logique du jeu en une architecture logicielle claire, modulaire et évolutive. Le jeu de dames, avec sa complexité stratégique, a constitué un bon support pédagogique pour appliquer les notions de classes, attributs, méthodes et gestion d'état. Développé initialement en version console, ce projet pourrait être étendu à une interface graphique pour enrichir l'expérience utilisateur.

Ce rapport retrace les étapes du projet, nos choix techniques, les difficultés rencontrées, et les perspectives d'amélioration.

II. Principe de jeu

Le jeu de dames oppose deux joueurs sur un plateau de 64 cases (8x8), où seules les cases foncées sont utilisées. Chaque joueur commence avec 12 pièces. Les pions se déplacent en diagonale vers l'avant d'une case à la fois. La capture des pièces adverses est obligatoire et se fait en sautant par-dessus, avec possibilité de prises multiples. Lorsqu'un pion atteint la dernière rangée adverse, il est promu en dame, bénéficiant ainsi d'une plus grande liberté de déplacement en diagonale dans toutes les directions sur plusieurs cases. La partie se termine lorsqu'un joueur n'a plus de pièces, est bloqué ou qu'un match nul est déclaré.

III. Méthodologie

1. Organisation et prise de décision collective

Au début du projet, une réunion d'équipe nous a permis de définir le design et les éléments de l'interface utilisateur et la structure des classes. Nous avons décidé des fonctionnalités clés, comme les effets sonores, l'affichage des déplacements possibles, la bordure rouge pour les pions obligés de capturer, et l'annonce du joueur gagnant. Nous avons également identifié les bibliothèques et outils nécessaires. Nous avons aussi décidé de travailler dans Google Colab pour faciliter la collaboration et le suivi des versions. Cette étape a aussi servi à clarifier nos objectifs, organiser les tâches et assurer la cohérence du projet.

2. Packages Java

Le package `java.awt` a été utilisé pour la gestion des composants graphiques et le dessin du plateau, à l'aide de classes telles que `Color`, `Graphics`, `Graphics2D`, `Dimension`, `Font` et `BorderLayout`. Pour la création de l'interface graphique, le package `javax.swing` a été employé, en s'appuyant sur des classes comme `JFrame`, `JPanel`, `JLabel` et `SwingUtilities`, permettant d'afficher les fenêtres, les panneaux et les éléments visuels du jeu.

Le package `java.awt.event` gère les interactions avec la souris, notamment grâce aux classes `MouseEvent` pour détecter les clics et `MouseAdapter`, qui simplifie la gestion des événements souris.

Le package `javax.sound.sampled` est utilisé pour la gestion des sons, avec les classes `AudioInputStream` pour lire les fichiers audio, `AudioSystem` pour accéder aux ressources audio, et `Clip` pour la lecture des effets sonores et le package `java.util` est utilisé pour gérer des structures de données dynamiques, telles que les listes de pions et de mouvements, grâce aux classes `ArrayList` et `List`.

3. Conception orientée objet et architecture MVC

Nous avons commencé par modéliser les différentes entités du jeu en classes Java cohérentes, avec une séparation claire des responsabilités selon le paradigme MVC (Modèle-Vue-Contrôleur):

Modèle (Model)

Le modèle contient la logique métier et l'état du jeu, avec des classes comme *Plateau* qui gère le damier 8x8. Les pièces sont représentées par une hiérarchie de classes (*Piece* abstraite, *Pion*, *Dame*), chacune définissant le comportement spécifique des pièces. La classe *Position* encapsule les coordonnées sur le plateau, tandis que les énumérations *Couleur* et *TypePiece* définissent respectivement la couleur des joueurs et les types de pièces.

Vue (View)

La vue est assurée par la classe *PlateauDeJeu*, qui hérite de *JPanel* et s'occupe d'afficher le plateau, les pièces, les coups possibles, les sélections ainsi que les messages à l'utilisateur

Contrôleur (Controller)

Le contrôleur, représenté par la classe *JeuDeDamesGUI*, initialise la fenêtre principale, configure l'interface graphique et gère les événements utilisateurs. Il utilise la classe *SoundManager* pour jouer les effets sonores liés aux événements du jeu, comme les déplacements ou la victoire.

Cette organisation modulaire améliore la lisibilité, la réutilisabilité et la testabilité du code.

IV. Description synthétique des classes

L'architecture du code repose sur une série de classes bien définies, chacune jouant un rôle précis dans la structure du jeu de dames.

JeuDeDamesGUI : L'interface principale

La classe *JeuDeDamesGUI* constitue l'interface principale du jeu. Elle gère la fenêtre graphique à travers un *JFrame* et centralise les interactions entre l'utilisateur, l'affichage, les messages de jeu et le suivi des tours. Par composition, elle possède un objet *PlateauDeJeu*, responsable de l'affichage du plateau, et applique l'encapsulation pour protéger ses composants graphiques. Son

rôle est de superviser le bon déroulement de la partie tout en déléguant les tâches spécifiques à d'autres classes spécialisées.

SoundManager : Le gestionnaire des effets sonores

SoundManager est une classe utilitaire centralisée chargée de gérer les sons du jeu, comme les captures et les victoires. Grâce à l'utilisation exclusive de méthodes statiques, elle permet d'appeler les effets sonores sans instancier d'objet, ce qui simplifie son usage et renforce le couplage faible avec les autres composants. L'abstraction est bien respectée, car les classes extérieures ne voient jamais les détails techniques du chargement des fichiers audio.

TypePiece et Couleur

Les énumérations TypePiece et Couleur forment un duo fondamental pour représenter les deux dimensions essentielles des pièces : leur nature (PION ou DAME) et leur appartenance (BLANC ou NOIR). Ces types sont utilisés dans toutes les classes manipulant des pièces et permettent de garantir la clarté du code tout en réduisant les erreurs liées aux valeurs arbitraires. Leur simplicité structurelle en fait des constantes partagées à travers le système.

Position : Localisation immuable des pièces

La classe Position représente une case sur le damier par deux coordonnées entières : ligne et colonne. Ces attributs sont finaux et définis uniquement dans le constructeur, rendant chaque objet Position immuable. Cette conception protège les coordonnées d'une modification accidentelle et assure la stabilité des références dans tout le système. Grâce à ses méthodes getLigne() et getColonne(), et à sa redéfinition de equals(), la classe offre une abstraction propre et sûre pour manipuler les positions sur le plateau.

Piece : Abstraction mère de toutes les pièces

La classe abstraite Piece définit le modèle de base de toutes les pièces du jeu. Elle possède deux attributs protégés, position et couleur, et déclare deux méthodes abstraites essentielles : getDeplacementsPossibles() et getPrisesPossibles(), que chaque sous-classe devra implémenter selon ses règles. Elle permet ainsi au système de traiter uniformément toutes les pièces, tout en laissant chaque sous-classe personnaliser son comportement.

Pion : La pièce de base du jeu

Pion est une sous-classe concrète de Piece qui implémente les règles spécifiques aux déplacements de base : diagonaux courts vers l'avant et prises par saut au-dessus d'une pièce adverse. Il réutilise les attributs hérités tout en ajoutant une logique propre. Ce comportement spécialisé respecte le principe d'héritage et démontre le polymorphisme dynamique.

Dame : La pièce promue et stratégique

La classe Dame, qui hérite également de Piece, redéfinit les comportements pour permettre des déplacements et des prises longues en diagonale, dans toutes les directions. Elle implémente aussi la logique des prises multiples, ce qui la rend plus complexe et stratégique que le Pion.

Plateau : Le cerveau logique du jeu

PlateauDeJeu : Le relais graphique entre le joueur et le jeu

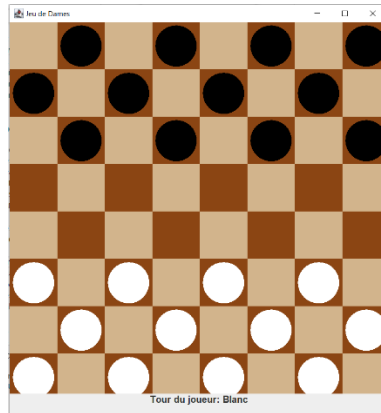
V. Le diagramme de classes UML de l'application



VI. Captures d'écran de l'exécution du jeu

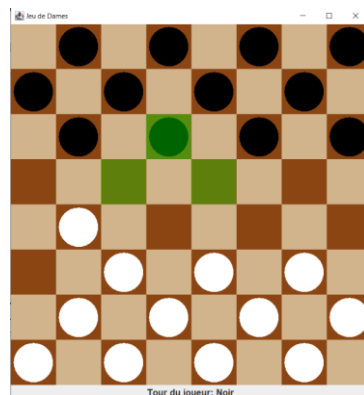
1- Plateau du jeu

Nous avons utilisé deux couleurs pour le plateau : marron et beige, et les pièces sont représentées par des cercles noirs et blancs.



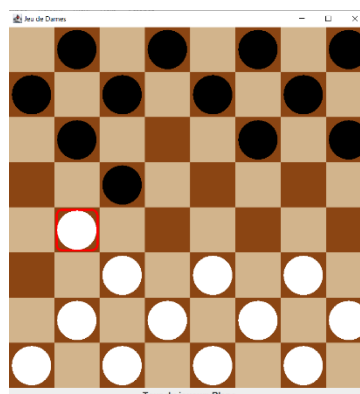
2- Déplacement des Pions

Les pions avancent uniquement vers l'avant, en diagonale, sur les cases foncées du plateau, avec une indication en vert des déplacements possibles pour chaque pion.



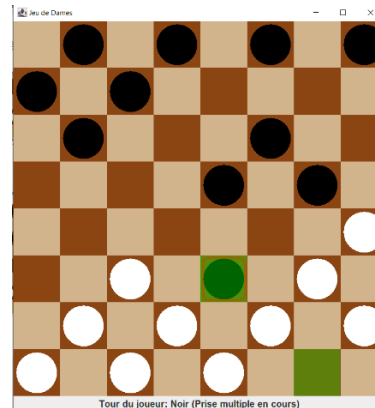
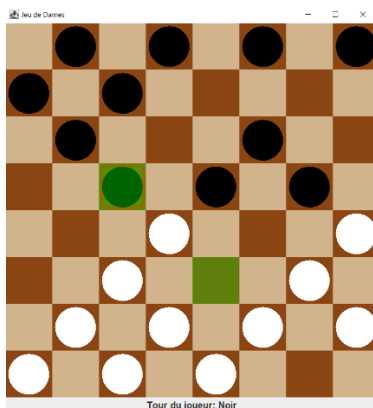
3- Capture Obligatoire

Lorsqu'une prise est possible, le joueur doit obligatoirement capturer la pièce adverse. Pour faciliter la sélection, le pion concerné est entouré d'une bordure rouge.



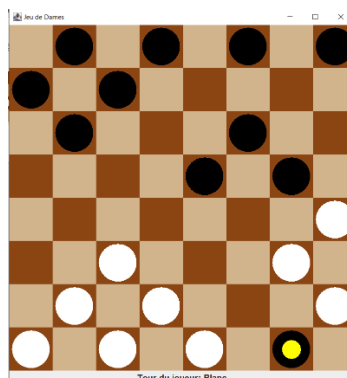
4- Prise multiple

Un pion peut effectuer plusieurs captures consécutives lors d'un même tour si les conditions de prise le permettent.



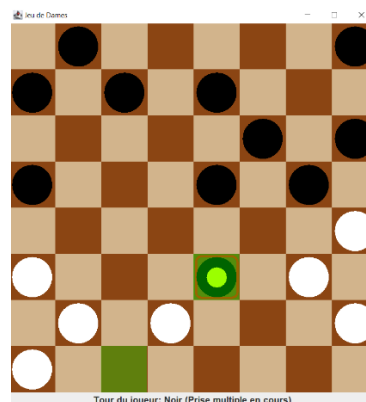
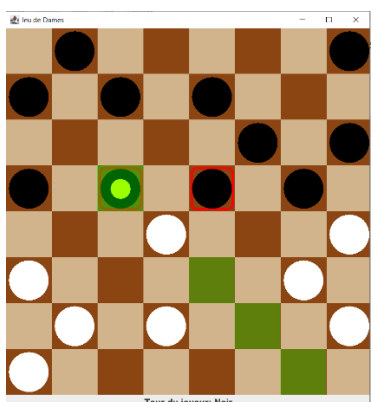
5- Promotion en Dame

Lorsqu'un pion atteint la dernière rangée adverse, il est promu en dame, gagnant ainsi de nouvelles capacités de déplacement.



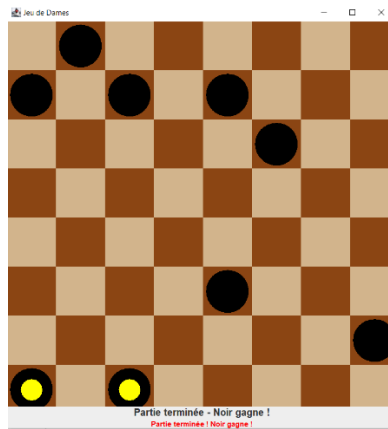
6- Prise par la Dame

La dame peut se déplacer librement en diagonale sur plusieurs cases et capturer les pièces adverses à distance selon les règles du jeu.



7- Fin partie

La partie prend fin quand un joueur ne peut plus déplacer ou capturer, et un message annonce alors le gagnant.



VII. Répartition des tâches entre les membres

Chaque membre a pris en charge un ensemble de classes spécifiques, de manière à favoriser un développement parallèle :

🚩 Membre 1 (Khaddor Douae) :

Ce membre a développé les classes Piece, Pion et Dame. Elle a aussi intégré les énumérations TypePiece(pion ou dame) et Couleur (blanc ou noir).

🚩 Membre 2 (Makhloufi Ouiam) :

Ce membre s'est chargé de la logique du jeu dans la classe Plateau. Elle a implémenté les règles principales, comme la validation des déplacements, la gestion des prises obligatoires, la promotion des pions en dames, et la détection de la fin de partie. Elle a également développé la classe Position qui représente les coordonnées des cases du plateau et sert de base pour appliquer ces règles.

🚩 Membre 3 (El Qasemy Niamat) :

Ce membre a créé l'interface graphique avec les classes PlateauDeJeu et JeuDeDamesGUI. Elle a géré les événements de la souris, l'affichage dynamique du plateau, des pièces, et des déplacements possibles. Elle a également développé la classe SoundManager.

VIII. Conclusion

Le développement de ce jeu de dames a représenté un véritable défi technique, notamment en raison de la complexité de la synchronisation entre la logique du jeu et l'interface graphique. La liaison entre les classes de la logique métier et celles de l'interface, a parfois posé des difficultés.

La gestion de certains cas particuliers, comme les prises multiples ou la promotion en dame, a également exigé une mise à jour précise et instantanée de la logique et de l'affichage.

Pour améliorer l'expérience utilisateur, plusieurs pistes sont envisageables : la création d'une interface plus moderne avec des menus et plusieurs pages, l'ajout d'un mode multijoueur en ligne pour jouer avec d'autres personnes, ainsi qu'un système de score pour rendre la compétition plus motivante. L'intégration d'un mode contre une intelligence artificielle permettrait aussi de jouer en solo, tandis qu'un minuteur offrirait la possibilité d'ajouter une contrainte de temps aux parties, pour plus de dynamisme et de challenge.

Enfin, ce projet nous a permis de développer nos compétences en Java et en programmation orientée objet. C'est un projet très amusant qui nous offre un jeu complet à utiliser pendant notre temps libre.